

Data Exchange Formats

This document contains useful information on how to parse data from various Data Exchange Formats.

Note: The terminology included in this document and a basic understanding of JSON, XML, and HTML is course content and therefore may be included on exams or quizzes.

Table of Contents

Introduction	2
Terminology	2
CSV / TSV	3
JSON	3
Purpose	3
Structure	3
Parsing	4
Reading JSON	4
Writing JSON	6
XML	7
Purpose	7
Structure	7
Parsing	9
HTML	11
Purpose	11
HTML vs XML	11
Structure	11
Parsing	14
Other Common Data Exchange Formats	14

Introduction

Imagine you have information about the world tour schedule of your favorite artist. You know exactly when your favorite artist will be coming to cities in the United States, where these concerts

will take place, and how much it will cost to go to these concerts. You want to deliver this information to your friends so that you all can go to the concerts together. How would you do this? Your first thought might be that “Oh, I can just send a message with all this information in your group chat.” That works, however the group chat could get very cluttered. Alternatively, you might call them one by one and deliver this information over the phone, however that may be time consuming if you have lots of friends.

In the two scenarios above, you are exchanging the data you have about concerts to your friends, whether it’s through the text message you send or through a phone call. This is an example of how we humans can exchange data, but how do computers exchange data with each other? If you have millions of data points that you want to exchange with another human, will the “human” way of exchanging data still work?

There are a number of existing data exchange formats, and we will only focus on the most common ones that industrial engineers frequently interact with. They all have their unique characteristics, advantages, and disadvantages, and it’s important that you know the differences.

Terminology

Data Exchange Format: an interim data format for converting from one file or database structure to another. The source data is converted into the exchange format by one program, and the data in the exchange format is converted to the target format by another program.

CSV/TSV - Comma-Separated-Value format/Tab-Separated-Value format

JSON - JavaScript Object Notation

XML - eXtensible Markup Language

HTML - HyperText Markup Language

ElementTree - A module for parsing XML code

Element - One node in a structured hierarchy such as that used by xml or html to store data.

Tag - Indicates the start or end of an xml or html element.

Attribute - A name/value pair that defines a property of an element.

CSV / TSV

Comma Separated Values (CSV) is a common data exchange format that is widely supported by consumer, business, and scientific applications for tabular data. The format is primarily used for data import/export and exchange in Spreadsheet applications and databases. Each record in a CSV file is contained in a single line where each field value is separated by a comma and each record is separated by a newline character. Tab separated values (TSV) format is less commonly used, but is similar to CSV except the delimiter is a tab character instead of a comma. More information about

CSV and TSV files and how to read and write them can be found in the File I/O and CSV Module handout.

JSON

Purpose

JSON stands for **JavaScript Object Notation**. It is a lightweight data-exchange format. JSON uses JavaScript syntax, but the JSON format is text only. Therefore, it is easy to manipulate JSON files in any programming language. It is widely used for web services responses.

Structure

Below is the JSON representation of “people.json”.

```
{
  "people": {
    "person": [
      {
        "firstName": "Alan",
        "lastName": "Turing",
        "professions": {
          "profession": ["Computer Scientist",
            "Mathematician", "Cryptographer"]
        }
      },
      {
        "firstName": "Stephen",
        "lastName": "Hawking",
        "professions": {
          "profession": ["Physicist", "Comedian"]
        }
      }
    ]
  }
}
```

JSON is very similar to a nested Python dictionary. However, there are some key differences:

1. JSON uses double quotes (") for all strings
2. JSON uses true and false for boolean values instead of True and False
3. JSON uses null instead of None
4. JSON dictionary keys must be string type

Parsing

We will use [Python's built-in JSON module](#) to convert a JSON file or response to a familiar data structure like a dictionary or list. In order to do this, we need to first, `import json`

Reading JSON

Two methods from the JSON module are used for reading in JSON data. The `loads` (load s) method parses a JSON string and turns it into a Python data structure, such as a Python dictionary or Python list. The `load` method parses a JSON file and turns it into a Python data structure, such as a Python dictionary or Python list.

- The **loads** method parses a string of JSON code and turns it into a Python dictionary

```
jsonstring2dict = json.loads('jsonstring')
```

```
>>> import json
>>> json_string = '{"Alan Turing":["Computer Scientist", "Mathematician",
"Cryptographer"], "Stephen Hawking":["Physicist", "Comedian"]}'
>>> output_dict = json.loads(json_string)
>>> output_dict
{'Alan Turing': ['Computer Scientist', 'Mathematician', 'Cryptographer'],
 'Stephen Hawking': ['Physicist', 'Comedian']}
>>> output_dict['Alan Turing']
['Computer Scientist', 'Mathematician', 'Cryptographer']
>>> output_dict.values()
dict_values(['Computer Scientist', 'Mathematician', 'Cryptographer'],
 ['Physicist', 'Comedian'])
```

Notice that in the example above, `output_dict` is just a regular Python dictionary that we can easily manipulate. Also note that since JSON requires double quotes around strings, we need to use single quotes to denote the entire JSON string.

- The **load** method translates the data in a JSON file into a Python dictionary.

```
with open('jsonfilename.json', 'r') as f:
```

```
    json2dict = json.load(f)
```

```
$ cat people.json # for windows use "type people.json"
{
  "people": {
    "person": [
      {
        "firstName": "Alan",
        "lastName": "Turing",
        "professions": {
          "profession": ["Computer Scientist", "Mathematician", "Cryptographer"]
        }
      },
      {
        "firstName": "Stephen",
        "lastName": "Hawking",
        "professions": {
          "profession": ["Physicist", "Comedian"]
        }
      }
    ]
  }
}
```

Load in data from json files by using the load method.

```
output_dict2 = json.load(open('people.json', 'r'))
```

```
>>> from pprint import pprint
>>> pprint(output_dict2)
{'people': {'person': [{'firstName': 'Alan',
                           'lastName': 'Turing',
                           'professions': {'profession': ['Computer Scientist',
                                                            'Mathematician',
                                                            'Cryptographer']}},
                       {'firstName': 'Stephen',
                           'lastName': 'Hawking',
                           'Professions': {'profession': ['Physicist',
                                                            'Comedian']}}]}}
>>> output_dict2['people']['person'][0]['firstName']
'Alan'
```

Again, notice that in the example above, `output_dict2` is just a regular Python dictionary that we can easily manipulate. To clearly visualize this dictionary, we can use the `pprint` module. `pprint` is a built-in module that prints out nested data structures in a visually-appealing way. Also note that the file name and the access mode need to be included when opening the file object.

Writing JSON

Two methods from the JSON module are used for writing JSON data. The `dumps` (dump s) method takes a Python Dictionary and returns a JSON string while the `dump` method takes a Python Dictionary and a file object in writing mode and writes the Python Dictionary to the file.

- The **dumps** method takes a Python dictionary and returns a JSON string.

```
dict2jsonstring = json.dumps(the_dict)
```

```
>>> output_dict
{'Alan Turing':['Computer Scientist', 'Mathematician', 'Cryptographer'],
 'Stephen Hawking':['Physicist', 'Comedian']}
>>> json_string = json.dumps(output_dict)
>>> json_string
'{"Alan Turing":["Computer Scientist", "Mathematician", "Cryptographer"],
 "Stephen Hawking":["Physicist", "Comedian"]}'
>>> type(json_string)
str
```

- The **dump** method takes a Python dictionary and dumps it into a JSON file.

```
with open('file_out.json', 'w') as f:
    json.dump(output_dict, f)
```

```
>>> output_dict
{'Alan Turing':['Computer Scientist', 'Mathematician', 'Cryptographer'],
 'Stephen Hawking':['Physicist', 'Comedian']}
>>> json.dump(output_dict, open('people_out.json', 'w'))
>>> cat people_out.json
{"Alan Turing":["Computer Scientist", "Mathematician", "Cryptographer"],
 "Stephen Hawking":["Physicist", "Comedian"]}
```

Notice that the file object is in writing mode, so the access mode when opening the file is indicated by the `'w'`.

XML

Purpose

XML stands for **eXtensible Markup Language**. It is both human and machine readable and is often used for distributing data over the Internet. It is a verbose textual representation of trees.

Structure

Below is a sample XML file named "people.xml".

```
<people>
  <person gender = "male">
    <firstName>Alan</firstName>
    <lastName>Turing</lastName>
    <professions>
      <profession>Computer Scientist</profession>
      <profession>Mathematician</profession>
      <profession>Cryptographer</profession>
    </professions>
  </person>
  <person gender = "male">
    <firstName>Stephen</firstName>
    <lastName>Hawking</lastName>
    <professions>
      <profession>Physicist</profession>
      <profession>Comedian</profession>
    </professions>
  </person>
</people>
```

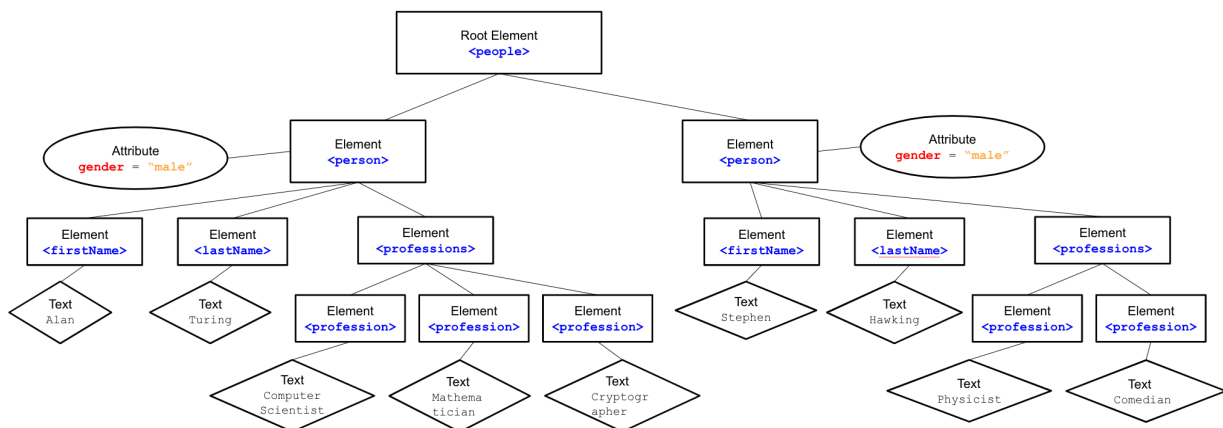
Blue: Tag

Red: Attribute

Orange: Attribute Value

Black: Text

This XML file can be visualized using a diagram to show the tree like nature of XML.



Things you need to know about the structure of XML:

- XML documents are formed as element trees. An XML tree starts at a **root element** and branches from the root to **child elements**.
- An XML tree must contain one and only one unique root element that is the parent of all other elements. For instance, `<people>` is the root element which has 2 direct child elements `<person>`.
- All elements can have sub elements (child elements). For instance, `<person>` further branches into `<firstName>`, `<lastName>`, and `<professions>`.
- Elements on the same level are considered siblings. For instance, `<firstName>`, `<lastName>`, and `<professions>` are all siblings to each other.
- Every XML element consists of a start/opening tag (e.g. `<profession>`) and an end/closing tag (e.g. `</profession>`), which is denoted by `/` followed by the tag name. Every start tag must be followed by an end tag. An XML element is **everything** from (including) the element's start tag to (including) the element's end tag. You can have an empty tag like this: `<tagName></tagName>`.
- All XML elements must be properly nested within each other. Make sure you always close the child tags before you close the parent tags. For instance, `<firstTag><secondTag>...</secondTag></firstTag>` is valid, whereas `<firstTag><secondTag>...</firstTag></secondTag>` is invalid.
- Attributes are designed to contain data related to a specific element. Attribute values must always be quoted. For instance, the `person` tag has a `gender` attribute with value `"male"`. There is no rule about when to use attributes and when to use elements in XML. So if we were to make the `gender` attribute a child element of `person`, the XML file would still contain the same information.
- Some characters have a special meaning in XML. For instance, `"<"` and `">"` denote the starting and ending of a tag. Therefore, we cannot use `"<"` to denote "less than" or `">"` to denote "greater than" in an XML element. For this reason, we have something called "entity references" in XML. There are 5 predefined entity references in XML:

Entity Reference	To Replace	Meaning
<code>&lt;</code>	<code><</code>	less than
<code>&gt;</code>	<code>></code>	greater than
<code>&amp;</code>	<code>&</code>	ampersand
<code>&apos;</code>	<code>'</code>	apostrophe
<code>&quot;</code>	<code>"</code>	quotation mark

For instance, we use `<` to denote the meaning of "less than":

```
<message>salary &lt; 1000</message>
```

Only `<` and `&` are strictly illegal in XML, but it is a good habit to replace `>` with `>` as well.

- Comments in XML are denoted by `<!-- comments go here -->`.

- Element naming rules:
 - Element names are case-sensitive. For instance, `<Person>` is not the same as `<person>`.
 - Element names must start with a letter or underscore
 - Element names cannot start with the letters xml (or XML, or Xml, etc)
 - Element names can contain letters, digits, hyphens, underscores, and periods
 - Element names cannot contain spaces
 - No other reserved words except for xml

Parsing

We use Python's built-in module [xml.etree.ElementTree](#) to parse XML data.

- First, we need to import the module. It's convention to import `xml.etree.ElementTree` as `ET`.

```
import xml.etree.ElementTree as ET
```

- Now we create our `ElementTree` object.

```
>>> tree = ET.parse("people.xml")
>>> tree
<xml.etree.ElementTree.ElementTree at 0x1070017f0>
```

- Define the root object.

```
>>> root = tree.getroot()
>>> root
<Element 'people' at 0x1070a0368>
```

- Now from the root of the `ElementTree`, we search for the child tags. There are two methods that we can call on an `Element` object to search for child tags.
 - `.findall('tagName')` returns a list of all `Element` objects named `tagName` or an empty list if `tagName` is not found.
 - `.find('tagName')` returns the first `Element` object named `tagName` or `None` if `tagName` is not found.
- Note that both methods above will only look for the direct child tags of a given `Element` object. In other words, to find the most inner element of an `ElementTree`, you need to go deeper into the `ElementTree` one level at a time.
 - Since `<person>` is the direct child element of the root element `<people>`, we can use `.findall('person')` to find all the `<person>` elements.

```
>>> person_list = root.findall('person')
>>> person_list
[<Element 'person' at 0x1070a0ae8>, <Element 'person' at 0x1070a07c8>]
```

- Or we can use `.find('person')` to find the first `<person>` element.

```
>>> first_person = root.find('person')
>>> first_person
<Element 'person' at 0x1070a0ae8>
```

- However, since `<profession>` and `<firstName>` are not direct child tags of the root element `<people>`, we cannot find these tags from root.

```
>>> root.findall('profession')
[]
>>> root.find('firstName')
```

- The root element does not have a child tag named `'profession'`, `'firstName'` or `'whatever'`, so the returned value will be empty if `findall` is used and `None` if `find` is used.

```
>>> root.find('whatever')
```

- After successfully finding an element object using `findall` or `find`, an `Element` object is returned. All `Element` objects have an attribute `text`, which contains the string representation of the text within the `Element` object. Note that in the example below, the text within the `firstName` tag and the `lastName` tag are printed.

```
>>> for person in person_list:
    print(person.find('firstName').text)
    print(person.find('lastName').text)
Alan
Turing
Stephen
Hawking
```

- Mixing it all together. Note how we first find `<person>`, then `<professions>`, then `<profession>`, and finally the text between `<profession>` in the example below. We go deeper into this `ElementTree` one level at a time.

```
>>> for person in person_list:
    professions = person.find('professions')
    profession_list = professions.findall('profession')
    for i in profession_list:
        print(i.text)
    print('\n')
Computer Scientist
Mathematician
Cryptographer

Physicist
Comedian
```

HTML

Purpose

HTML stands for **HyperText Markup Language**. It is a standard markup language for Web pages. It describes the structure of a Web page. Browsers use HTML tags to render the content of the page. Web browsers such as Chrome, FireFox, and Safari read HTML documents and display them.

HTML vs XML

- XML is for data exchange/storage and HTML is mostly for data display.
- XML is both human and machine readable whereas an HTML file rendered in a web browser is human-readable.
- XML is case sensitive and HTML is not case sensitive.
- XML tags are user defined and HTML tags are predefined.
- In XML, closing tags are necessary. In HTML, closing tags are not necessary. HTML also has tags such as the image tag that can be opened and closed at the same time.

Basics

Some common HTML tags::

- **Setup tags**
 - All HTML documents must start with a document type declaration:
`<!DOCTYPE html>`
 - The HTML document itself begins with `<html>` and ends with `</html>`
 - The visible part of the HTML document is between `<body>` and `</body>`

Example:

```
<!DOCTYPE html>
<html>
```

```
<body>
    <!-- Content here -->
</body>
</html>
```

- **HTML Headings**

Headings are defined with the <h1> to <h6> tags, with <h1> being the most important and the biggest header and <h6> being the least important and the smallest header.

Example:

```
<h1> This is the biggest header </h1>
<h2> This is a smaller header </h2>
<h6> This is the smallest header </h6>
```

After being rendered, the tags above will produce something like this in a Web page:

This is the biggest header

This is a smaller header

This is the smallest header

- **HTML paragraphs**

HTML paragraphs are defined with the <p> tag.

Example:

```
<p>This is a paragraph.</p>
```

- **HTML Links**

HTML links are defined with the <a> tag.

Example:

```
<a href="https://www.google.com">Click Here</a>
```

This will highlight “Click Here” and allow you to click on it.

After being rendered, the tags above will produce something like this in a Web page:

[Click Here](https://www.google.com)

- **HTML Images**

HTML images are defined with the tag.

Example:

```
<img src =
"https://www.python.org/static/opengraph-icon-200x200.png">
```

After being rendered, the tags above will produce an image of Python in a Web page:



- **HTML Lists**

HTML lists are defined with the `<ul>` (unordered/bullet list) or the `<ol>` (ordered/numbered list) tag, followed by `<li>` tags (list items):

Example1:

```
<ul>
  <li> Do homework </li>
  <li> Do Laundry </li>
  <li> Eat </li>
</ul>
```

After being rendered, the tags above will produce an image of Python in a Web page:

- Do homework
- Do Laundry
- Eat

Example2:

```
<ol>
  <li> Do homework </li>
  <li> Do Laundry </li>
  <li> Eat </li>
</ol>
```

After being rendered, the tags above will produce an image of Python in a Web page:

1. Do homework
2. Do Laundry
3. Eat

- **HTML Tables**

An HTML table is defined with the `<table>` tag. Each table row is defined with the `<tr>` tag. A table header is defined with the `<th>` tag. By default, table headings are bold and centered. A table data/cell is defined with the `<td>` tag.

```
<table>
  <tr>
    <th>First Name</th>
    <th>Last Name</th>
    <th>Professions</th>
```

```

</tr>
<tr>
    <td>Alan</td>
    <td>Turing</td>
    <td>Computer Scientist, Mathematician, Cryptographer</td>
</tr>
<tr>
    <td>Stephen</td>
    <td>Hawking</td>
    <td>Physicist, Comedian</td>
</tr>
</table>

```

After being rendered, the tags above will produce an image of Python in a Web page:

First Name	Last Name	Professions
Alan	Turing	Computer Scientist, Mathematician, Cryptographer
Stephen	Hawking	Physicist, Comedian

- **HTML Comments**

HTML comments will not be displayed by the browser, but will help you document your HTML file. They follow the syntax below:

```
<!-- Write your comments here -->
```

There is a lot more to HTML than what we have just introduced. However, you are only expected to understand the content above for this class. More on HTML here: [HTML Tutorial](#)

Rendering HTML

To render an HTML document we can utilize Python's built-in [http](#) library.

1. Launch the command line
2. Navigate to the directory containing the HTML file
3. Execute `python -m http.server`
4. Open a browser
5. Type <http://localhost:8000/> or <http://0.0.0.0:8000/> into the URL search bar
6. Press enter

Parsing

Parsing HTML can be a much more involved process that is out of scope for this document. Please reference the Requests, RegEx, and BeautifulSoup handout for further information.

Other Common Data Exchange Formats

Other interesting, although less commonly used data exchange formats have been developed. Among these are YAML (which stands for YAML Ain't Markup Language and which was the precursor to JSON), Gellish (a formalized subset of natural English with the concepts arranged in a subtype-supertype hierarchy), and RDF, (which stands for Rich Data Format and which is used by the semantic web to identify the meanings of terms).